

MODULE 4

Syllabus

Syntax directed translation - Syntax directed definitions, S-attributed definitions, L-attributed definitions, Bottom-up evaluation of S-attributed definitions. **Run-Time Environments - Source Language issues, Storage organization, Storage-allocation strategies. Intermediate Code Generation - Intermediate languages, Graphical representations, Three-Address code, Quadruples, Triples.**

Run Time Environments

- Need to relate static source text of program to actions that must occur at run-time to implement the program.
- The program consists of names for procedures, identifiers, etc., that require mapping with the actual memory location at runtime.
- Runtime environment is the state of the target machine, which may include software libraries, environmental variables etc to provide services to the processes running in the system.
- Allocation and deallocation of data objects is managed by runtime support package.

Source Language Issues

The program is made up of procedures.

1. Procedures

- A program is made up of procedures.
- A procedure definition is a declaration that associates an identifier with a statement.
- The identifier is the procedure name and the statement is the procedure body.

Eg: square(x:int) { x * x; }



procedure name procedure body

- Procedures that return values are called functions in many languages.
- A complete program will also be treated as a procedure.

- Procedure call: appearance of a procedure name in an executable statement.

- Procedure call executes procedure body.

```
val a = square(5);
```

- Formal parameter: identifier used to pass values into a procedure

- fun square(x:int) {x * x};

- Actual parameter : value passed to procedure in a call. They are substituted for the formals in the body.

- val a = square(5);

Procedure definition

procedure name
associated with
procedure body

```
PROGRAM sort(input,output);  
  VAR a : array[0..10] of Integer;  
  PROCEDURE readarray;  
    VAR i : Integer;  
    BEGIN  
      for i:= 1 to 9 do read(a[i]);  
    END;  
  FUNCTION partition(v,z : Integer): Integer;  
    VAR i,j,x,v : Integer;  
    BEGIN  
      ...  
    END;  
  PROCEDURE quicksort(m,n : Integer);  
    VAR i : Integer;  
    BEGIN  
      if (n > m) then BEGIN  
        i := partition(m,n);  
        quicksort(m, i-1);  
        quicksort(i+1,n)  
      END  
    END;  
  BEGIN /* of main */  
    a[0] := -9999; a[10] := 9999;  
    readarray;  
    quicksort(1,9)  
  END.
```

Formal parameters

Procedure call

Actual parameters

2. Activation Tree

- **Control flows sequentially**, each execution of the procedure starts at the beginning of the procedure body.
- Each execution of procedure body is referred to as an **activation of procedure**.
- **Lifetime of an activation** of a procedure p is the sequence of steps between the first and last statements in the execution of the procedure body, including time spent executing other procedures called by p .
- Lifetimes of two procedures are either non overlapping or nested. So we can represent procedure activations as a tree.
- Each time control enters a procedure q from procedure p , it eventually returns to p (if no errors occur)
 - Activation of procedure p ; Activation of procedure q ; Return to activation of procedure p .

- A procedure is Recursive if a new activation can begin before an earlier activation of the same procedure has ended.
- We can use a tree, called **the activation tree to depict the way control enters and leaves activations.**
- **How to build an activation tree:**
 - Each node represents an activation of a procedure.
 - The root represents the activation of the main program.
 - The node for a is the parent of node b if and only if control flows from activation a to b.
 - The node for a is to the left of the node for b if and only if the lifetime of a occurs before lifetime of b

EXAMPLE

Consider the following program of quicksort

```
main()
```

```
{
```

```
    readarray();
```

```
    quicksort(1,10);
```

```
}
```

```
quicksort(int m, int n)
```

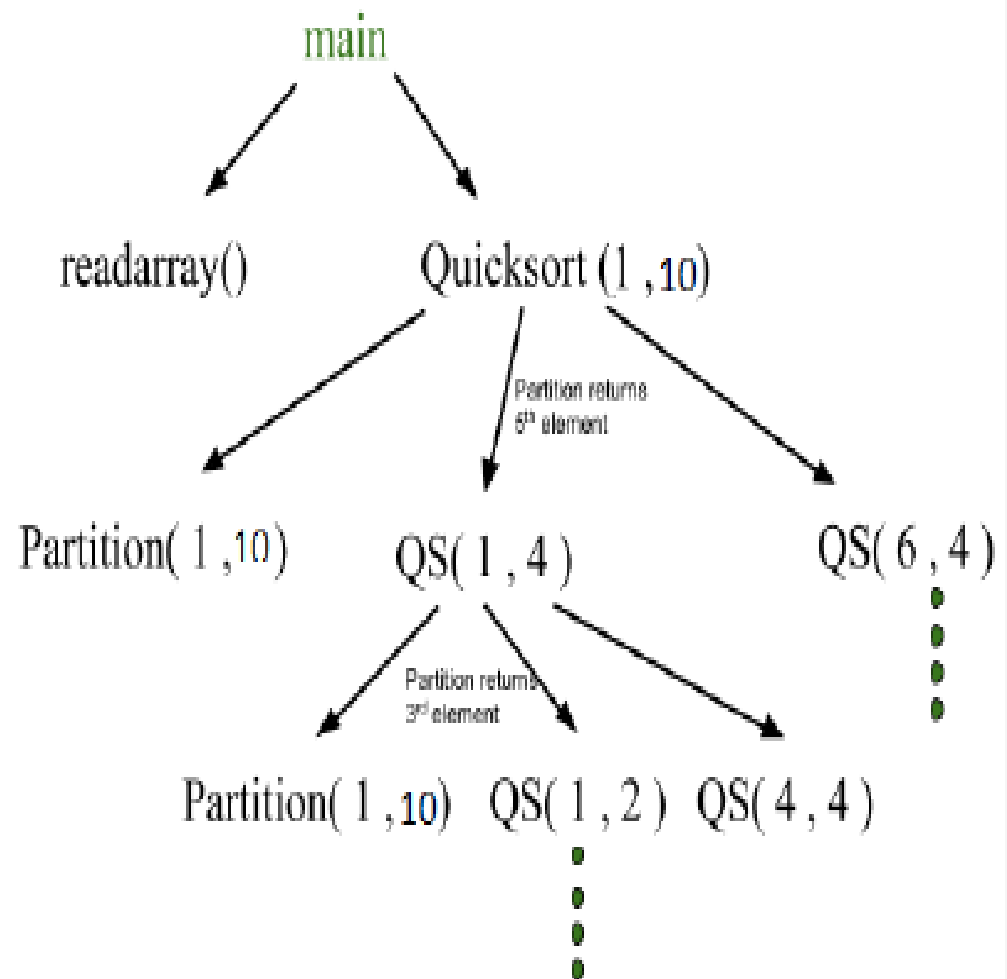
```
{
```

```
    int i= partition(m,n);
```

```
    quicksort(m,i-1);
```

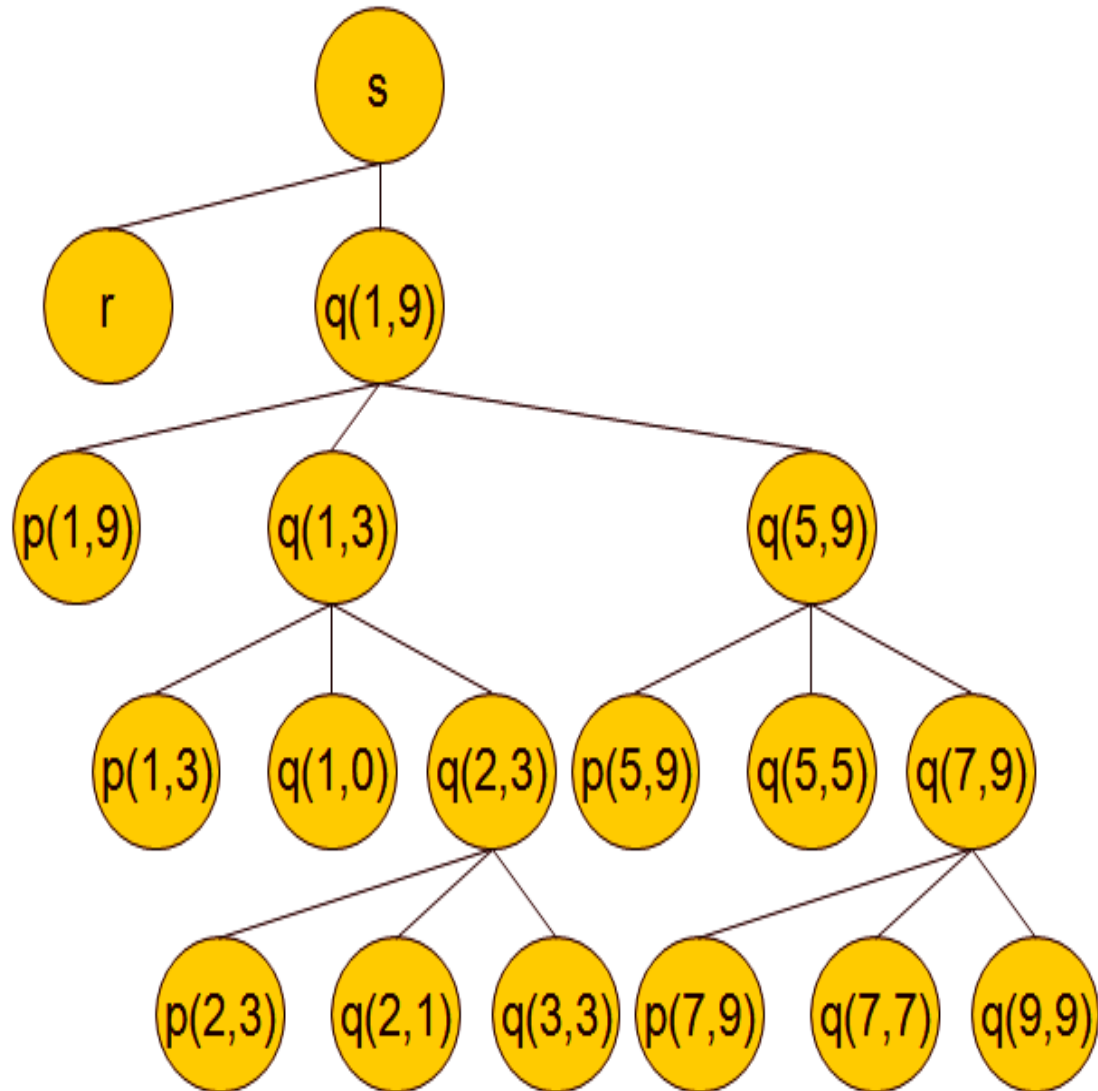
```
    quicksort(i+1,n);
```

```
}
```



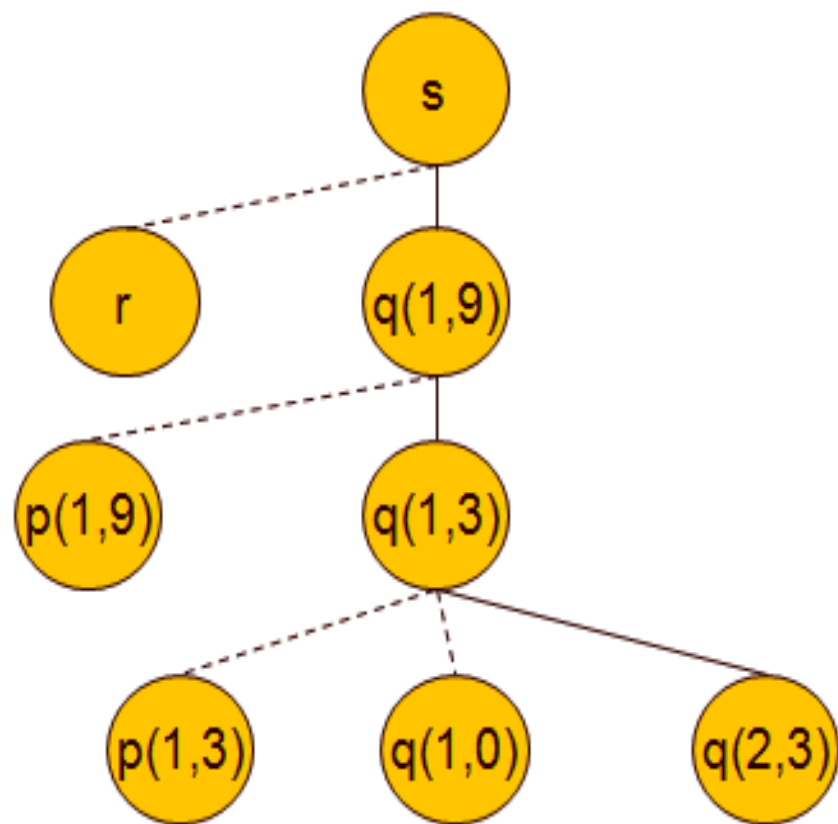
Eg: Activation Tree for a program

```
PROGRAM sort(input, output);  
  VAR a : array[0..10] of Integer;  
  PROCEDURE readarray;  
    VAR i : Integer;  
    BEGIN  
      for i:= 1 to 9 do read(a[i]);  
    END;  
  FUNCTION partition(y,z : Integer): Integer;  
    VAR i, j, x, v : Integer;  
    BEGIN  
      ...  
    END;  
  PROCEDURE quicksort(m,n : Integer);  
    VAR i : Integer;  
    BEGIN  
      if (n > m) then BEGIN  
        i := partition(m,n);  
        quicksort(m, i-1);  
        quicksort(i+1,n)  
      END  
    END;  
  BEGIN /* of main */  
    a[0] := -9999; a[10] := 9999;  
    readarray;  
    quicksort(1, 9)  
  END.
```



3. Control Stack

- Flow of control corresponds to depth-first traversal of the activation tree that starts from root, visits a node before its children, and recursively visits children at each node from left to right.
- Control stack keeps track of “live” (currently executing) procedure activations.
- Push the node for an activation onto the control stack as the activation begins and pop node when activation ends.
- Thus control stack are related to paths to the root of the activation tree.
- When node n is at the top of control stack, the stack contains nodes along the path from n to the root.



r , $p(1,9)$, $p(1,3)$ and $q(1,0)$
have executed to completion
(dashed lines)

Stack contains

$q(2,3)$ ← top
 $q(1,3)$
 $q(1,9)$
 s

Fig: Control Stack contains nodes along a path to the root.

4. Scope of Declarations:

- A Declaration in a Language is a syntactic construct that associates information with a name.

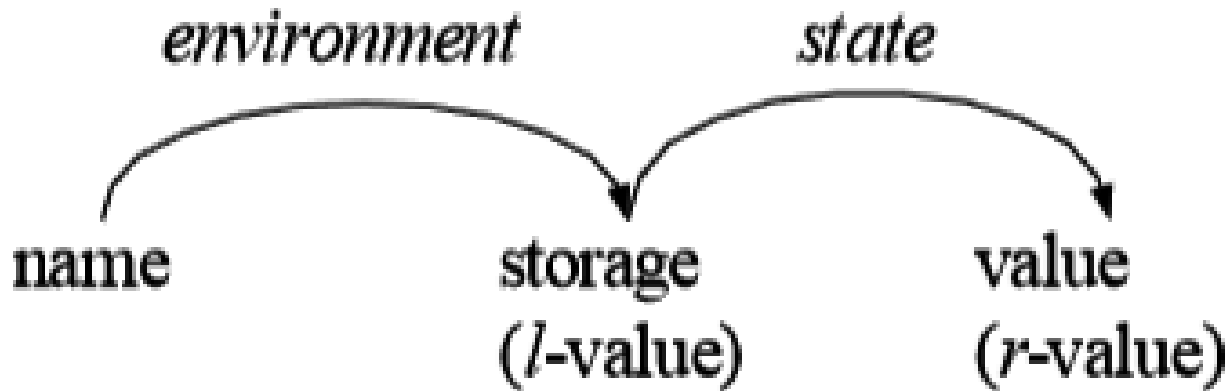
var i : integer

- Portion of program to which declaration applies is called scope of declaration.
- There are two types of scope: Local and Nonlocal.
- Occurrence of a name in a procedure is said to be **local** to a procedure if it is in the scope of procedure's declaration.
- Otherwise it is said to be **nonlocal**.

- At compile time, symbol table can be used to find the declaration that applies to an occurrence of a name.
- When a declaration is seen, symbol table entry is created for it.
- As long as we are in the scope of the declaration, its entry is returned when the name in it is looked up.

5. Binding of Names

- A given name in a program may denote different data objects at run time.
- Data object corresponds to a storage location that can hold values.
- The term **environment** refers to a function that **maps a name to a storage location (l-values)**
- The term **state** refers to a function that **maps a storage location to the value held there (r-values)**

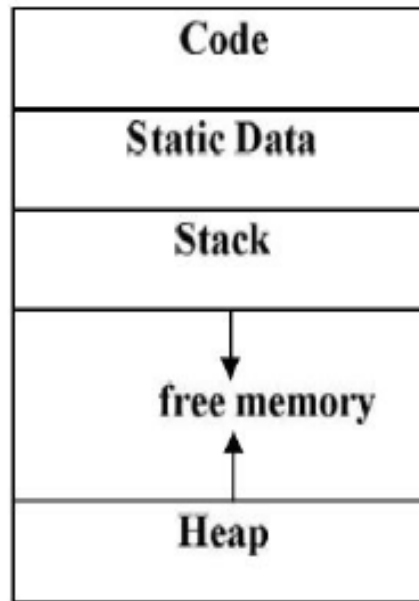


- An assignment statement changes state, not environment.
Eg: name: pi storage address: 100 value of pi=0
pi=3.14; // After execution value of pi=3.14 but storage address is still 100.
- Binding is association of name to storage location by environment.

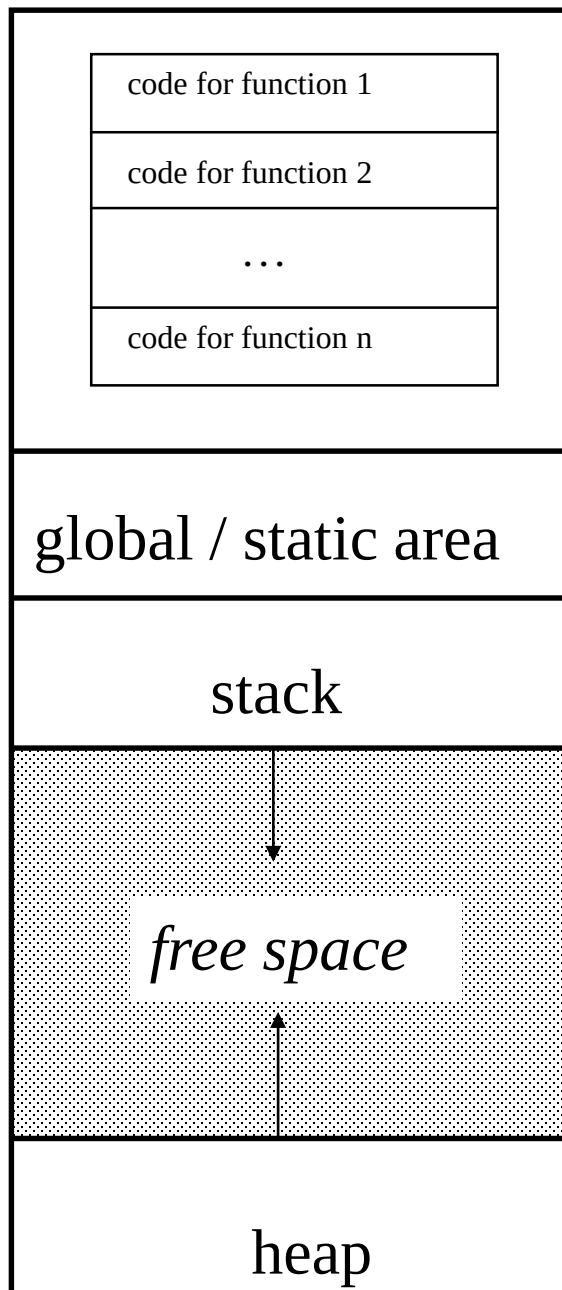
Storage Organization

- Compiler obtains a block of storage from operating system for the compiled program to run in.
- Run time storage must be divided to hold:
 - Generated target code
 - Data objects – corresponds to a storage location that can hold values.
 - Information to keep track of procedure activation.

Typical subdivision of run time memory.



- ✚ **Code area:** used to store the generated executable instructions, memory locations for the code are determined at compile time
- ✚ **Static Data Area:** Is the locations of data that can be determined at compile time
- ✚ **Stack Area:** Used to store the data object allocated at runtime. eg. Activation records
- ✚ **Heap:** Used to store other dynamically allocated data objects at runtime (for ex: malloc)



- The size of target code is fixed at compile time.
- Also size of certain data objects are also known in advance.
- PASCAL and C use extensions of the control stack to manage activations of procedures
- Stack contains information about register values, value of program counter and data objects of an activation which is interrupted to execute another.
- Heap holds all other information. For example, activations that cannot be represented as a tree.
- By convention, stack grows down and the top of the stack is drawn towards the bottom(value of top is usually kept in a register)

Activation Record(AR)/ Stack frame

- Procedure calls and returns are usually managed by a run time stack called control stack.
- Each live activation of a procedure has an activation record on control stack.
- Information needed for a single execution of a procedure is managed using a contiguous block of storage called an activation record or stack frame.
- The activation record gets pushed for each procedure call and popped for each procedure return.

- **AR consist of:**

return value
actual parameters
[optional] control link
[optional] access link
saved machine status
local data
temporaries

- **Temporaries:**

- hold temporary values that arises in the evaluation of expressions.

- **Local data:**

- holds data that is local to an execution of a procedure.

- **Saved machine status:**

- holds information about the state of machine just before the procedure is called.

- This information includes values of program counter and machine registers that have to be restored when control returns from procedure.

return value
actual parameters
[optional] control link
[optional] access link
saved machine status
local data
temporaries

- **Optional Access Link:**

- Refers to nonlocal data held in other Activation Record.

- **Optional Control Link:**

- points to the Activation Record of caller.

- **Actual Parameters:**

- used by the calling procedure to supply parameters to called procedure.

- **Returned value:**

- used by the called procedure to return value to calling procedure.

Storage Allocation Strategies

- The different Storage Allocation Strategies are:
 - **Static Allocation**
 - **Stack Allocation**
 - **Heap Allocation**
- Static allocation lays out storage for all data objects at compile time.
- Stack allocation manages run-time storage as a stack.
- Heap allocation allocates and deallocates storage as needed at runtime from a data area called heap.

1. Static Allocation

- Names are bound to storage at compile time.
- Since **bindings do not change at run time**, every time a procedure is activated, its names are bound to the same storage location.
- This property **allows values of local names to be retained across activations of a procedure.**
- That is, when control returns to a procedure, the values of the locals are the same as they were when control left last time.

- The compiler uses the type of a name (retrieved from the symbol table) to determine storage size required.
- The required number of bytes is set aside for the name.
- The address of the storage is fixed at compile time.
- **Limitations:**
 - The size required must be known at compile time.
 - Recursive procedures cannot be implemented because all activations of a procedure use the same bindings for local names.
 - Data structures cannot be created dynamically, since there is no mechanism for storage allocation at run time.

2. Stack Allocation

- **Dynamic allocation** based on the idea of a control stack.
- Storage is organized as a stack.
- Activation records are pushed to and popped from stack as activations begin and end.
- Storage for the locals in each call of a procedure is contained in the activation record for that call.
- This means **locals are bound to fresh storage on every call**, because a new AR is pushed onto the stack when a call is made.
- The values of the locals are deleted when the activation ends, because AR is popped.

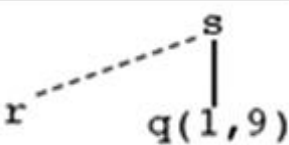
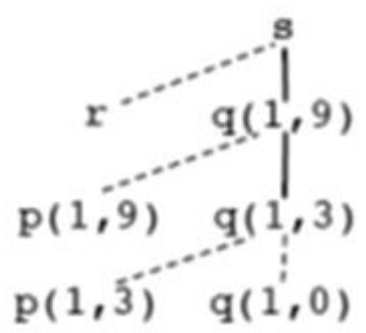
- ***top*** marks the top of the stack.
- To allocate a new activation record, just increment ***top*** by the size of record.
- To deallocate an existing activation record, just decrement ***top*** by the size of record.
- Eg: if procedure q has an activation record of size a, then top is incremented by 'a' just before the target code of q is executed. And when control returns from q, top is decremented by 'a'.

```

PROGRAM sort(input, output);
  VAR a : array[0..10] of Integer;
  PROCEDURE readarray;
    VAR i : Integer;
    BEGIN
      for i:= 1 to 9 do read(a[i]);
    END;
  FUNCTION partition(y,z : Integer): Integer;
    VAR i, j, x, v : Integer;
    BEGIN
      ...
    END;
  PROCEDURE quicksort(m,n : Integer);
    VAR i : Integer;
    BEGIN
      if (n > m) then BEGIN
        i := partition(m,n);
        quicksort(m, i-1);
        quicksort(i+1,n)
      END
    END;
  BEGIN /* of main */
    a[0] := -9999; a[10] := 9999;
    readarray;
    quicksort(1, 9)
  END.

```

Example:

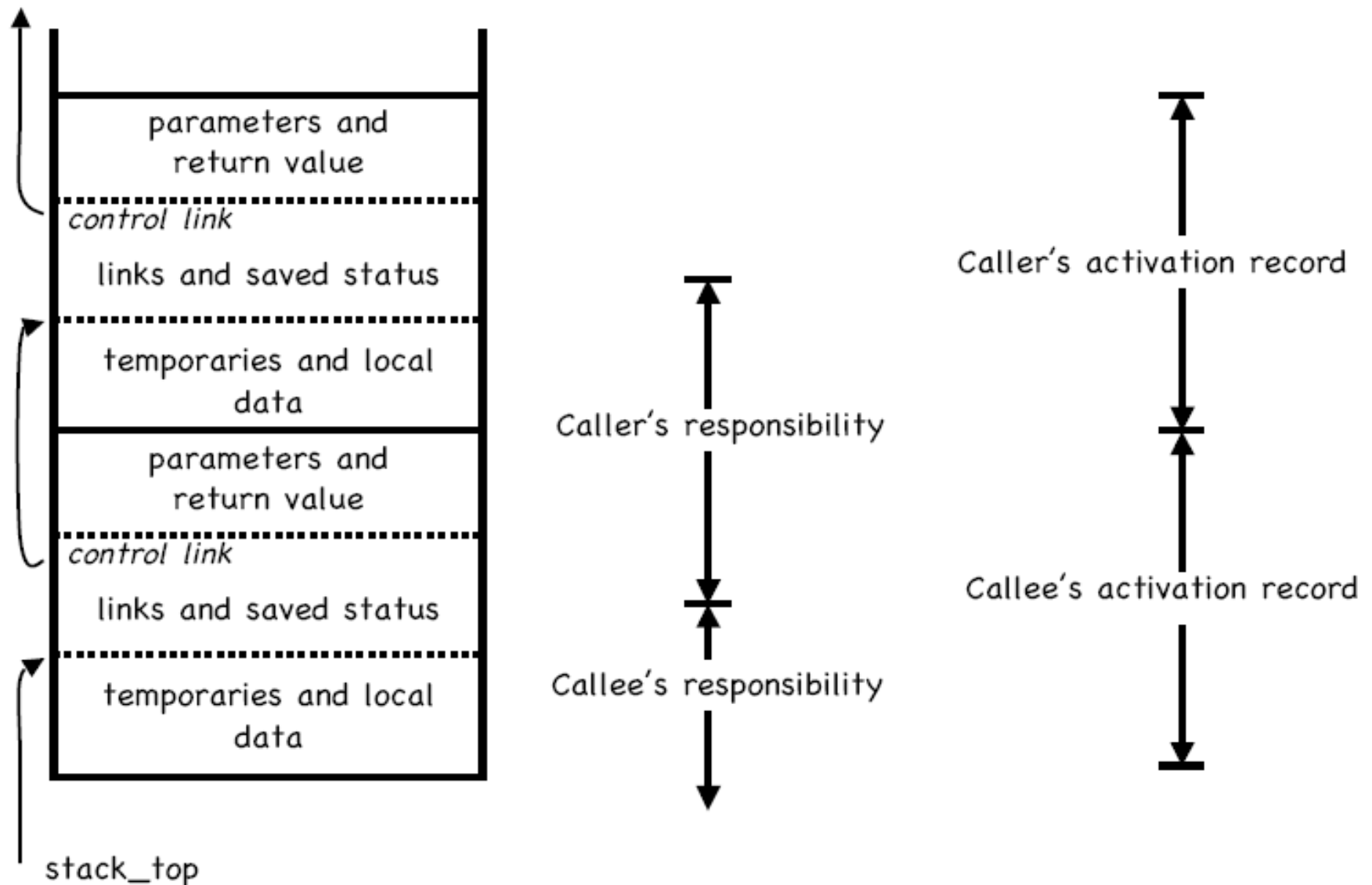
Position in Activation Tree	Activation Records on the Stack	Remarks
s	s a : array	Frame for s
	s a : array r i : integer	r is activated
	s a : array q(1,9) i : integer	Frame for r has been Popped and q(1,9) pushed.
	s a : array q(1,9) i : integer q(1,3) i : integer	Control has just returned to q(1,3)

Dashed line in tree represents activations that are ended.

Calling and Return Sequences

- Procedure calls are implemented by generating calling sequences in the target code.
- The **CALLING SEQUENCE** for a procedure **allocates an activation record and enters information into its fields.**
- The **RETURN SEQUENCE** **restores the machine state to allow execution of the calling procedure to continue.**
- The code in a calling sequence is often divided between the calling procedure (**the caller**) and the procedure it calls(**the callee**).

Division of task between caller and callee:



Sample calling sequence:

1. Caller evaluates the actual parameters and places them into the activation record of the callee.
2. Caller stores a return address and old value for `stack_top(top_sp)` in the callee's activation record.
3. Caller increments `stack_top` to the beginning of the temporaries and locals for the callee.
4. Caller branches to the code for the callee.
5. Callee saves all needed register values and status.
6. Callee initializes its locals and begins execution.

Sample return sequence:

1. Callee places the return value at the correct location in the activation record next to caller's activation record.
2. Callee uses status information previously saved to restore `stack_top` and the other registers.
3. Callee branches to the return address previously requested by the caller.
4. Caller copies the return value into its own activation record and uses it to evaluate an expression.

Dangling references:

- Whenever storage can be deallocated, the problem of dangling references arises.
- A dangling reference occurs when there is a reference to storage that has been deallocated.
- It is a **logical error** to use dangling references since the value of deallocated storage is undefined according to the semantics of most languages.

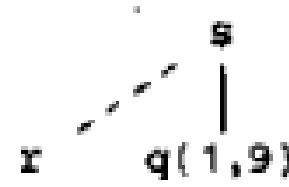
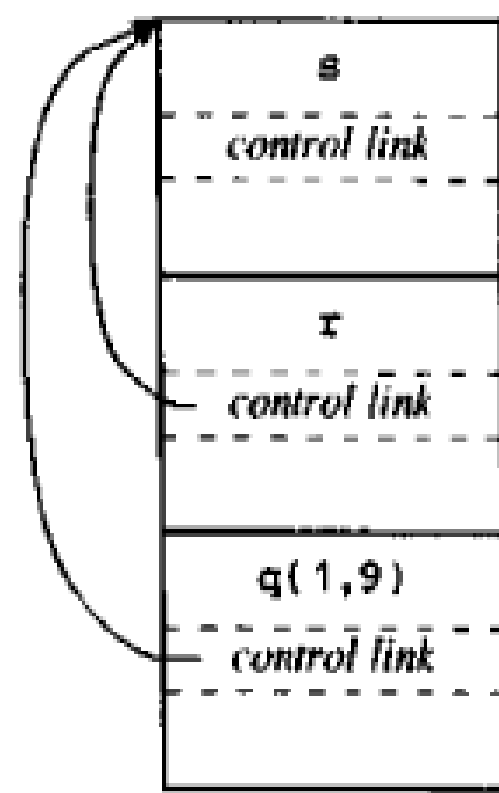
```
Main()
{
    int *p;
    p=dangle();
}
int *dangle()
{
    int i=23;
    return &i;
}
```

3. Heap Allocation

- Stack Allocation cannot be used when:
 1. Values of local names must be retained when an activation ends.
 2. A called activation outlives the caller (not common)In above cases, deallocation of AR need not be in LIFO order.
So storage cannot be organized as stack.

- Heap Allocation:

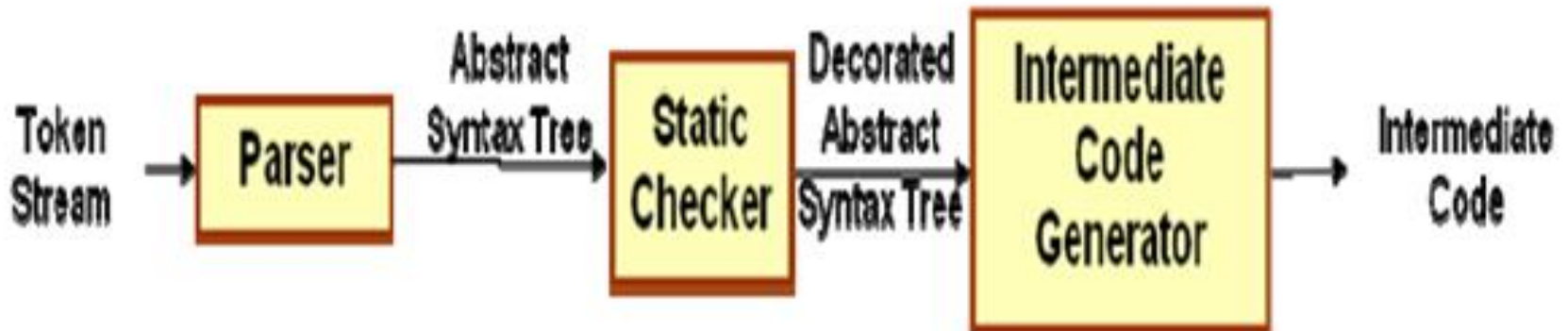
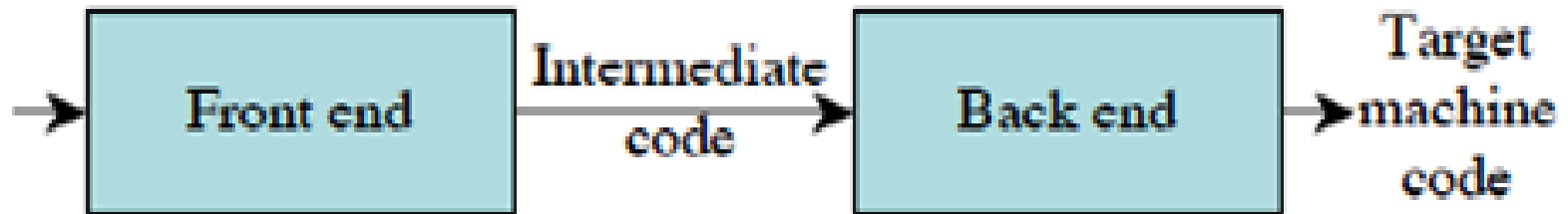
- Allocates pieces of contiguous storage, as needed for AR or other objects.
- Pieces may be deallocated in any order.
- So over time heap will consist of alternate areas that are free and in use. It is left to the heap manager to make use of this space.

POSITION IN THE ACTIVATION TREE	ACTIVATION RECORDS IN THE HEAP	REMARKS
 Diagram of activation tree: s is the parent of $q(1,9)$ (dashed line). x is a sibling of $q(1,9)$.	 Diagram of activation records in the heap: Three records are stacked vertically: 1. Top record: s, control link points to x. 2. Middle record: x, control link points to $q(1,9)$. 3. Bottom record: $q(1,9)$, control link field is present but not linked.	Retained activation record for x

Records for live activations need not be adjacent in a heap.

Intermediate Code Generation

- In compiler, the front-end translates a source program into an intermediate representation from which the back end generates target code.



Need for ICG.

1. If a compiler translates the source language to its target machine language without generating IC, then for each new machine, a full native compiler is required.
 2. A machine Independent Code-Optimizer can be applied to the Intermediate Representation.
- Two important things:
 - IC Generation process should not be very complex.
 - It should not be difficult to produce the target program from the intermediate code.

Intermediate Languages

- There are three kinds of Intermediate Representations:
 1. Graphical Representation
 - Syntax Tree
 - DAG (Direct Acyclic Graph)
 2. Postfix Notation
 3. Three Address Code

Graphical Representations

I) Syntax Tree or Abstract Syntax Tree(AST)

- Graphical Intermediate Representation.
- Syntax Tree depicts the hierarchical structure of a source program.
- Syntax tree (AST) is a condensed form of parse tree useful for representing language constructs.
- In a syntax tree, operators and keywords do not appear as leaves, but appears as intermediate nodes.

Parse tree and syntax tree for $3 * 5 + 4$ as follows.

Grammar

$E \rightarrow E + T$

$E \rightarrow E - T$

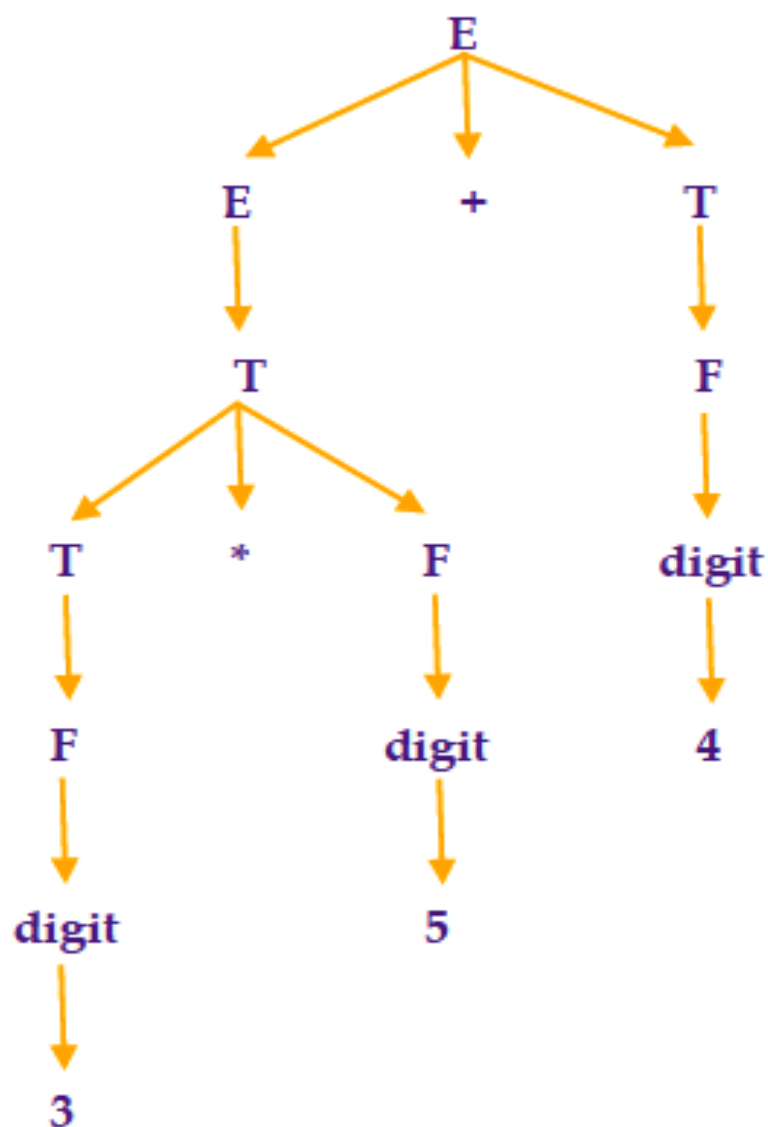
$E \rightarrow T$

$T \rightarrow T * F$

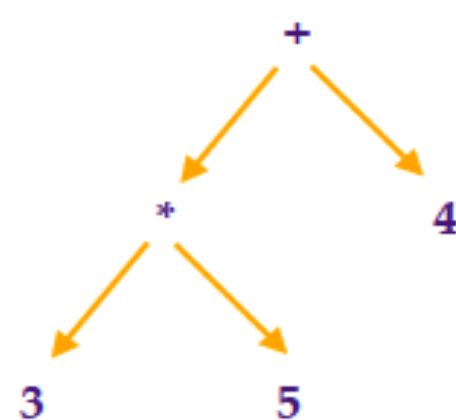
$T \rightarrow F$

$F \rightarrow \text{digit}$

Parse Tree



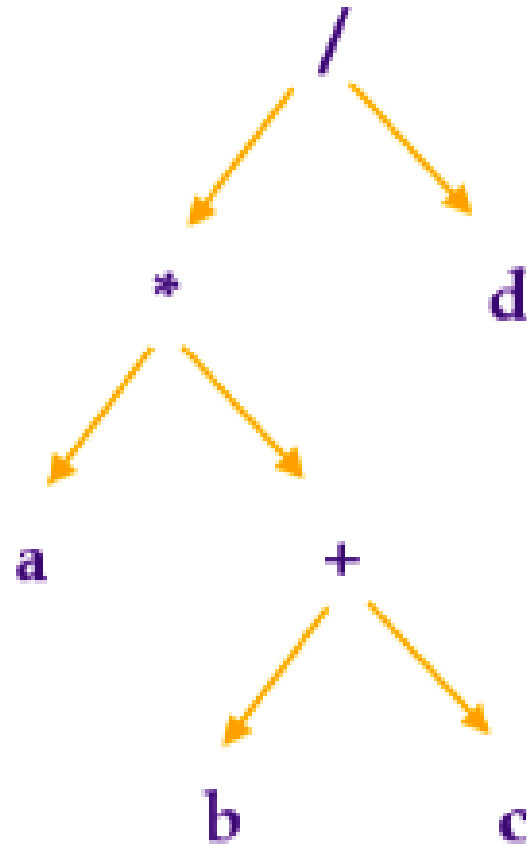
Syntax Tree



Parse Tree VS Syntax Tree

<u>Parse Tree</u>	<u>Syntax Tree</u>
A parse tree is a graphical representation of a replacement process in a derivation	A syntax tree (AST) is a condensed form of parse tree
Each interior node represents a grammar rule	Each interior node represents an operator
Each leaf node represents a terminal	Each leaf node represents an operand
Parse tree represent every detail from the real syntax	Syntax tree does not represent every detail from the real syntax Eg : No parenthesis

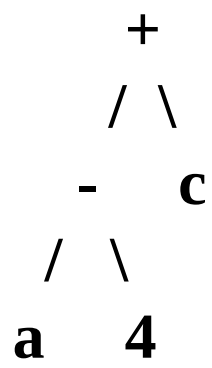
Syntax tree for $a * (b + c) / d$



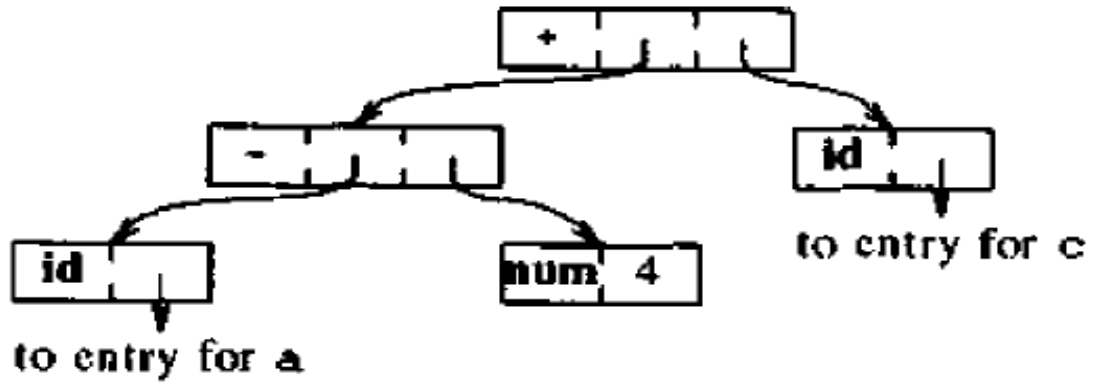
- Syntax tree can be represented in memory in 2 ways:
 1. Node Representation
 2. Tabular Representation

1. Node Representation

- Each node in a syntax tree can be implemented as a record with several fields.
- In the **node for an operator**, one field identifies the operator and the remaining fields contains the pointers to the nodes for operands.



Syntax tree



Syntax tree for **a-4+c**.

- Following **functions** are used to create the nodes of syntax tree:

1. ***mknode(op,left,right)***: creates an **operator node** with label ***op*** and two fields containing pointers to ***left*** and ***right***.
2. ***mkleaf(id,entry)***: creates an **identifier node** with label ***id*** and a field containing ***entry***, a pointer to the symbol table entry for identifier.
3. ***mkleaf(num,val)***: creates a **number node** with label ***num*** and a field containing ***val***, the value of the number.

EXAMPLE

$$a - 4 + c$$

The tree is constructed
bottom up

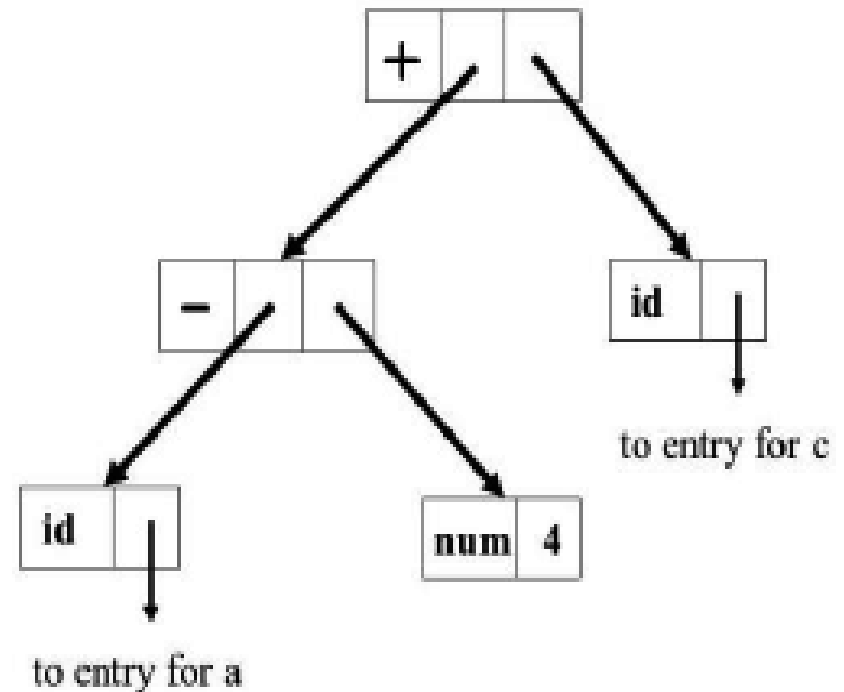
$P_1 = \text{mkleaf}(\text{id}, \text{entry } a)$

$P_2 = \text{mkleaf}(\text{num}, 4)$

$P_3 = \text{mknode}(-, P_1, P_2)$

$P_4 = \text{mkleaf}(\text{id}, \text{entry } c)$

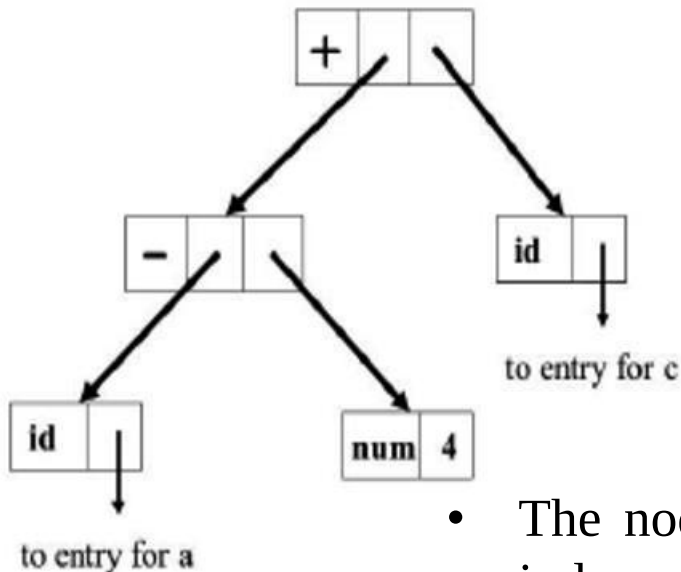
$P_5 = \text{mknode}(+, P_3, P_4)$



Syntax Tree

2. Tabular Representation

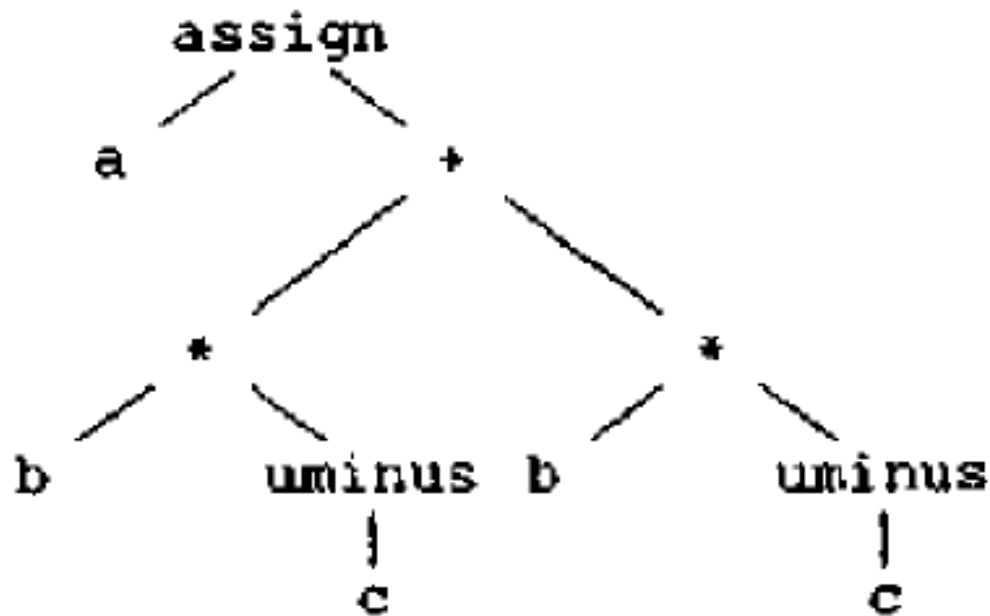
- In this, the nodes of tree are represented in a table format.
- The tabular representation for the syntax tree $a-4+c$ is:



0	id	a	
1	num	4	
2	-	0	1
3	id	c	
4	+	2	3

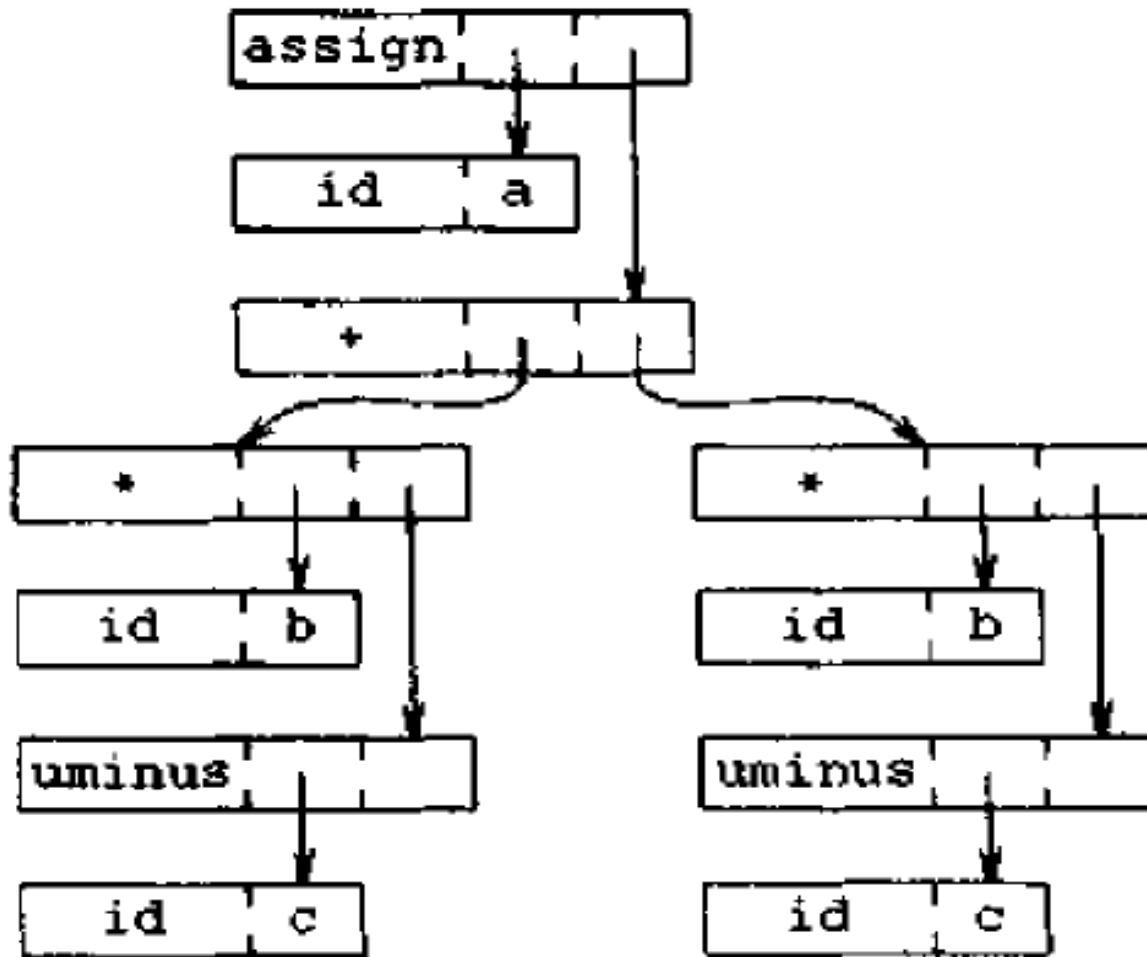
- The nodes are allocated from an array of records and the index or position of the node serves as the pointer to the node.
- All the nodes in the syntax tree can be visited by following pointers starting from the root at position 4.

Question: Draw a syntax tree for the expression $a = b * -c + b * -c$. Describe how it can be represented in memory?



(a) Syntax tree.

Node Representation of Syntax Tree



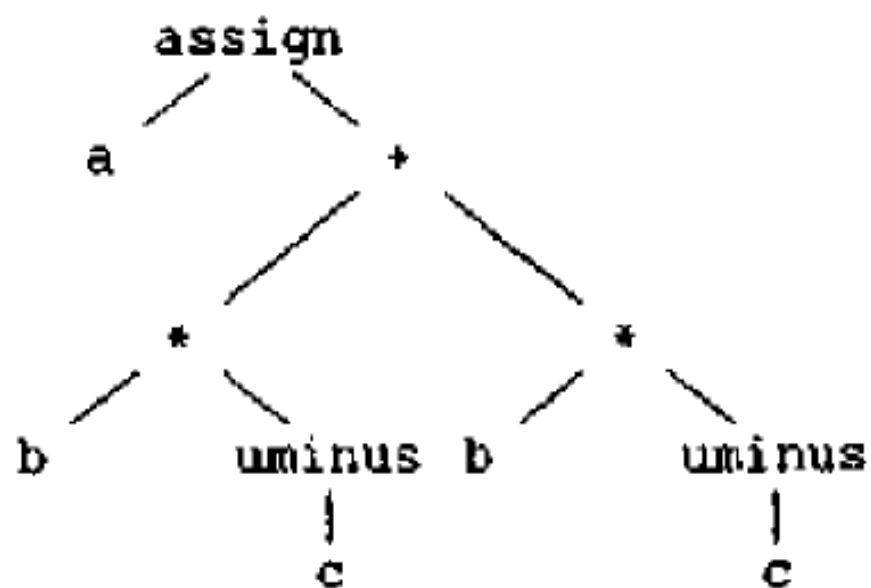
(a)

0	id	b	
1	id	c	
2	uminus	1	
3	*	0	2
4	id	b	
5	id	c	
6	uminus	5	
7	*	4	6
8	+	3	7
9	id	a	
10	assign	9	8

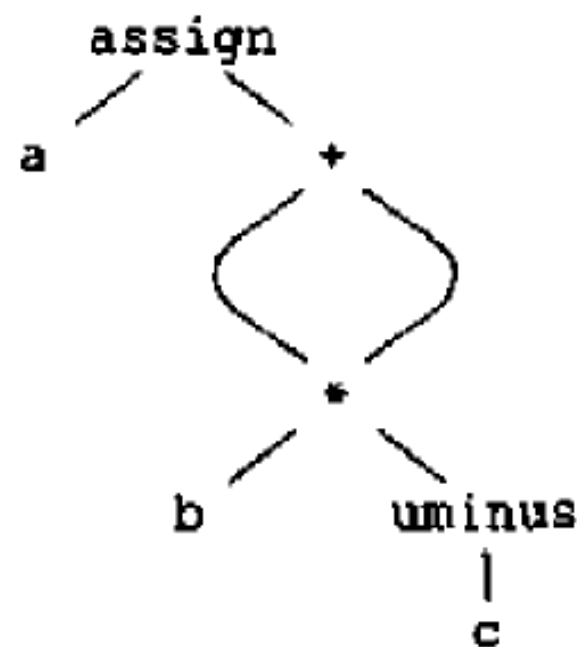
- The above fig is **tabular representation** of syntax tree.
- Here, nodes are allocated from array of records and the index or position of the node serves as the pointer to the node.

II) DAG - Direct Acyclic Graph.

- Graphical Intermediate Representation.
- DAG also gives the hierarchical structure of source program but in a more compact way because common sub expressions are identified.
- A DAG is obtained if the function constructing a node first checks to see whether an identical node already exists.
- If so return a pointer to the previously constructed node.



(a) Syntax tree.



(b) Dag.

Graphical representations of $a := b * -c + b * -c$.

Postfix Notation

- Linearized representation of syntax tree.
- In postfix notation, each operator appears immediately after its last operand.
- Operators can be evaluated in the order in which they appear in the string
- Ex:

Source String : **a := b * -c + b * -c**

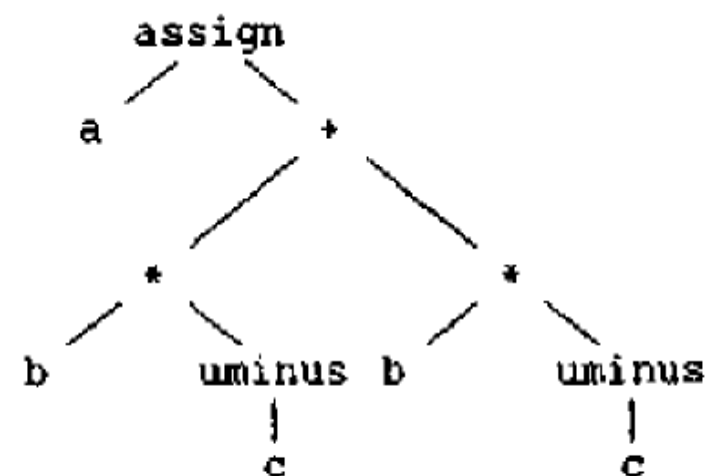
Postfix String: **a b c uminus * b c uminus * + assign**

Postfix Rules:

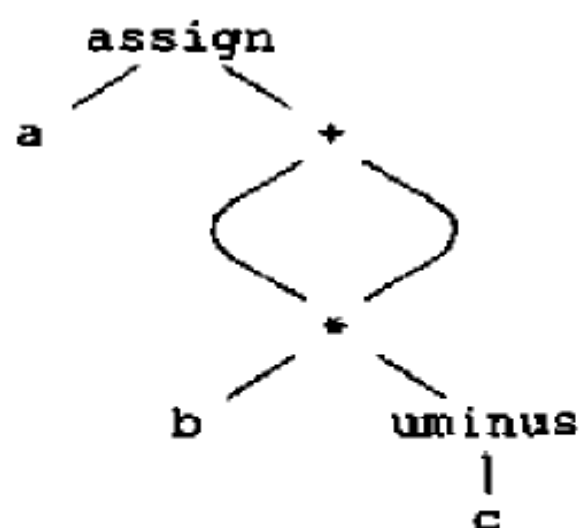
1. If E is a variable or constant, then the postfix notation for E is E itself.
 2. If E is an expression of the form $E_1 \text{ op } E_2$ then postfix notation for E is $E_1' E_2' \text{ op}$, here E_1' and E_2' are the postfix notations for E_1 and E_2 , respectively.
 3. If E is an expression of the form (E), then the postfix notation for E is the same as the postfix notation for E.
 4. For unary operation $-E$ the postfix is $E-$
Ex: postfix notation for $9 - (5+2)$ is **952+-**
- Postfix notation of an infix expression can be obtained using stack.

Three Address Code (TAC)

- 3-Address code is a sequence of statements of the general form:
 $x = y \text{ op } z$, where x, y, z are names, constants or compiler generated temporaries and op stands for any operator such as arithmetic operator or logical operator or boolean operator.
- The reason for the term “three address code” is that each statement contains at most 3 addresses. Two for the operands and one for the result.
- TAC is a Linearized representation of syntax tree.
- Ex: A source language expression like $x + y * z$ is translated into 3 Address code form:
 $t1 = y * z$
 $t2 = x + t1$ here $t1$ and $t2$ are compiler generated temporary names.



(a) Syntax tree.



Dag.

```

t1 := - c
t2 := b * t1
t3 := - c
t4 := b * t3
t5 := t2 + t4
a := t5
  
```

Code for the syntax tree.

```

t1 := - c
t2 := b * t1
t5 := t2 + t2
a := t5
  
```

Code for the dag.

Types of Three-Address Statements

1. *Assignment statements*

- $x := y \text{ op } z$, where op is a binary arithmetic or logical operation.

2. *Assignment instructions*

- $x := \text{op } y$, where op is a unary operation. Unary operations include unary minus, logical negation, shift operators etc.

3. *Copy statements*

- $x := y$, where the value of y is assigned to x.

4. *Unconditional jump*

- **goto L**, TAC with Label L is next to be executed.

5. *Conditional jump*

- **if x relop y goto L** This operation applies a relational operator(<,=,>=,etc) to x and y and executes L if true.

6. *Procedural call and return*

- param x; call p,n ; return y
- Procedure $p(x_1, x_2, \dots, x_n)$ is implemented in TAC as:
param x1
param x2
.....
param xn
call p , n

7. *Indexed Assignments*

$x = y[i]$ or $x[i] = y$

8. *Address and pointer assignments*

- $x = \&y$, $x = *y$, $*x = y$

Advantages of three-address code:

- Complex arithmetic expressions and nested flow of control statements can be written using series of three-address code statements. It is easier to optimize statements and generate target code from three-address code.
- The use of names for the intermediate values computed by a program allows three address code to be easily rearranged – unlike postfix notation.

Implementation of Three-Address Statements

- In compiler, three address statements are implemented as records with fields for the operator and operands.
- 3 such representations are:
 1. Quadruples
 2. Triples
 3. Indirect Triples

1. Quadruples

- In the quadruple representation, there are four fields for each instruction:
 - *op, arg1, arg2, result*
- op field represents the operator. arg1 and arg2 represents the arguments of operation.
- Unary ops does not use arg2. Operators like param neither use arg2 nor result.
- Conditional and unconditional jumps put the target label in result.
- Since 4 fields are used, it is called as quadruples.

- The quadruples in Fig (b) implement the three-address code in (a) for the expression

$$a = b * -c + b * -c$$

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

```

(a) Three-address code

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

(b) Quadruples

- The contents of fields `arg1`, `arg2` and `result` are normally pointers to the symbol table entries for the names represented by these fields. Therefore, temporary names must be entered into symbol table as they created.

2. Triples

- A triple has only three fields for each instruction: *op*, *arg1*, *arg2*
- The result of an operation $x \text{ } op \text{ } y$ is referred to by its position.
- To avoid entering temporary names into the symbol table, we refer a temporary value by the position of statement that computes it.
- Since 3 fields are used, it is called as triples
- Sometimes called as “Two-address code”.

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

Quadruples

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

- Parenthesized numbers represent pointers into the triple structure, while symbol table pointers are represented by the names themselves.

3. Indirect Triples

- This representation is an enhancement over triples representation.
- Consist of listing pointers to triples, rather than listing the triples themselves.
- With indirect triples, an optimizing compiler can move an instruction by reordering the instruction list, without affecting the triples themselves.

instruction

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
		...	

Figure 6.12: Indirect triples representation of three-address code

TUTORIAL

1. Construct the syntax tree and then draw the DAG for the statement:

$$e := (a*b) + (c-d) * (a*b)$$

2. Construct the DAG and three address code for the expression
 $a+a*(b-c)+b*(b-c)+b$

3. Write quadruples, triples and indirect tuples for the expression
 $(a + b) * (b + c) + (a + b + c)$